

Weekly Report #3

Benchmarking Stream Clustering Algorithms in River

Contributor	Nguyen Van A	Reporting period	08 Jun 2026 – 14 Jun 2026
Mentors	Dr. B, Dr. C	Phase	Phase 1 — Benchmark foundation
Repository	github.com/user/repo	Report date	September 11, 2026

1. Executive Summary

Objective — Establish the core benchmark metrics suite for stream clustering, so that every algorithm added later can be scored with one unified, reproducible interface (maps to Phase 1 — Benchmark foundation, milestone P1.2).

Status — **DONE On Track** — the metric selection and the evaluation protocol were finalised on schedule; the calculation script is 60% implemented and no blocker has appeared.

Progress tracker

Table 1: Weekly plan versus reality, week 3.

Weekly plan item	Phase ref.	Status	Completion
Select metrics suite (ARI, purity, silhouette, throughput)	P1.2	DONE	100%
Write metric calculation script	P1.2	IN PROGRESS	60%
Draft benchmark pipeline architecture	P1.3	DONE	100%
Unit tests for incremental silhouette	P1.2	PLANNED	0%

Takeaway. Only the testing task slipped to next week; it is not on the critical path because the pipeline skeleton (P1.3) does not depend on it.

Overall: Week 3/18 — on schedule.

2. Completed Tasks

- C1. Surveyed the internal design of River’s clustering module** (P1.1)
Summarised how `CluStream` and `DenStream` are implemented in River and identified **3 structural differences**: (i) micro-cluster storage (fixed array vs. dynamic dict), (ii) decay handling (time-window vs. exponential fading), and (iii) the absence of a common `predict_one` contract for outlier points. *Evidence:* [notes/river-internals.md](#), [commit a1b2c3d](#).
- C2. Finalised the metric suite and its justification** (P1.2)
4 metrics selected out of 11 candidates; each one mapped to a concrete failure mode it detects (see Table 2). *Evidence:* [PR #12](#).
- C3. Implemented MetricRegistry and 2 of 4 metrics** (P1.2)
ARI and purity run incrementally over a stream of 100k points in **1.9 s** with a constant memory footprint of ≈ 12 MB. *Evidence:* [bench/metrics.py](#).

In progress

- IN PROGRESS** **Incremental silhouette** (60%) — the sampling-based approximation works,

but the error bound versus the exact score is still unmeasured. Expected completion: Mon of week 4.

- **PLANNED Unit tests + CI hook** (0%) — moved to next week, deliberately, to keep the metric API stable first.

Time log

Research River internals: **6h** · Metric selection & write-up: **3h** · Implement metric script: **4h** · Concept-drift reading: **2h** · Mentor sync & report: **1.5h** ⇒ **Total: 16.5h**.

3. Results & Visualization

3.1 Benchmark pipeline architecture

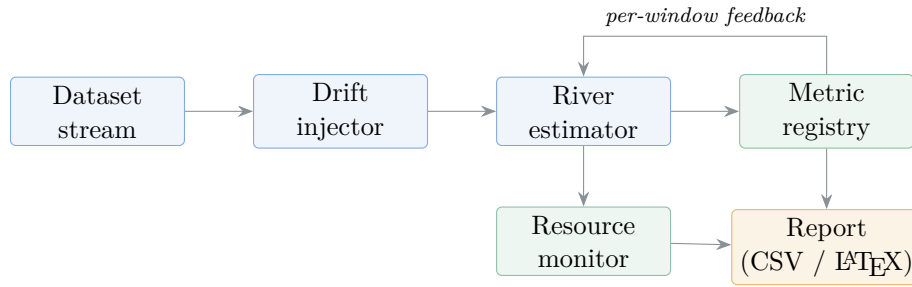


Figure 1: Proposed benchmark pipeline. Each block is a swappable component behind a single interface.

Takeaway. Decoupling the drift injector from the estimator means the same algorithm can be re-run under different drift regimes without touching the scoring code, which is what makes the benchmark reproducible.

3.2 Metric definitions

The internal quality of a partition is measured with the silhouette score. For a point i with mean intra-cluster distance $a(i)$ and mean nearest-cluster distance $b(i)$:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad s(i) \in [-1, 1], \quad (1)$$

and the score of the stream window W is the running mean $\bar{s}_W = \frac{1}{|W|} \sum_{i \in W} s(i)$.

External agreement uses the adjusted Rand index, which corrects the raw Rand index for chance:

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}} \quad (2)$$

where n_{ij} is the contingency count, a_i and b_j the row/column sums.

Table 2: Selected metrics and the reason each one is included.

Metric	Type	Range	Why it is in the suite
ARI (Eq. 2)	External	$[-1, 1]$	Chance-corrected; comparable across datasets with different cluster counts.
Purity	External	$[0, 1]$	Cheap to compute online; exposes majority-class collapse that ARI can hide.
Silhouette (Eq. 1)	Internal	$[-1, 1]$	The only metric usable when ground-truth labels are unavailable.
Throughput	System	pts/s	Stream algorithms are useless if they cannot keep up with the arrival rate.

Takeaway. The suite deliberately mixes external, internal and system metrics: an algorithm that wins on ARI but processes 200 points/s would fail the real use case, and the table makes that trade-off visible in one place.

3.3 Single figure


Figure 2: Clustering result of CluStream on the **sea** dataset after 50k points.

Takeaway. Micro-clusters stay well separated in the first two dimensions, which confirms the radius parameter chosen in week 2 is still appropriate at this stream length.

3.4 Side-by-side comparison


Figure 3: Cluster layout before and after an abrupt concept drift, same algorithm and same parameters.

Takeaway. The right-hand panel shows two micro-clusters that were never merged after the drift, i.e. the fading factor reacts too slowly — this is the motivation for issue I1 below.

3.5 Figure next to a table



Figure 4: Memory profile over the stream.

Table 3: Peak resource usage.

Algorithm	Peak RAM	pts/s
CluStream	41 MB	12 400
DenStream	78 MB	6 900
STREAMKMeans	19 MB	21 800

Takeaway. *DenStream costs roughly twice the memory of CluStream for a lower throughput; unless its quality advantage is large, it is the weakest candidate for the memory-constrained scenario in the proposal.*

3.6 Text wrapped around a figure

Use this layout when the figure is small and the surrounding explanation is what actually matters. Figure 5 tracks the mean micro-cluster radius across the stream; the radius grows monotonically until the first merge event, then stabilises. The important detail is the plateau after $t = 30k$: it means the structure has converged and any later change is caused by drift rather than by initialisation noise. Keep at least four or five lines of text next to a wrapped figure, otherwise the layout looks broken and it is better to fall back to a normal centred figure.



Figure 5: Micro-cluster radius growth.

3.7 Full result table

Table 4: Benchmark results on 3 datasets. Best value per column in **bold**.

Algorithm	sea		covtype		insects	
	ARI	Sil.	ARI	Sil.	ARI	Sil.
CluStream	0.71	0.55	0.63	0.48	0.52	0.41
DenStream	0.64	0.49	0.58	0.44	0.61	0.47
STREAMKMeans	0.58	0.61	0.55	0.46	0.44	0.38

Mean of 5 runs, window = 1000, seed $\in \{1..5\}$. Std. dev. < 0.03 everywhere.

Takeaway. *No algorithm dominates on all datasets: CluStream leads on the two synthetic streams while DenStream wins on the real, noisier **insects** data, which suggests noise robustness — not raw accuracy — is the property worth isolating in week 4.*

3.8 Performance chart

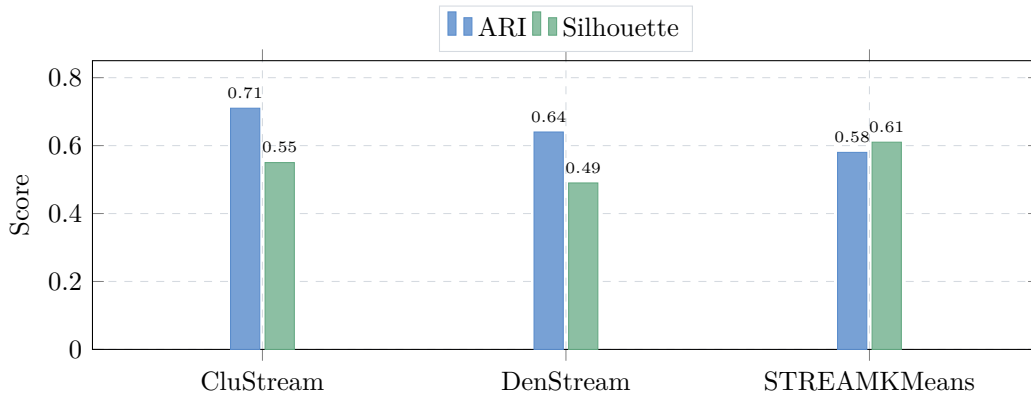


Figure 6: External (ARI) versus internal (silhouette) agreement on the `sea` dataset.

Takeaway. `STREAMKMeans` has the best silhouette but the worst ARI — a textbook sign that it produces geometrically tidy clusters that do not match the true labels, so silhouette alone must never be used as the selection criterion.

3.9 Reference implementation snippet

Listing 1: Incremental metric update loop.

```

1 for x, y_true in stream:
2     y_pred = model.predict_one(x)           # score before learning
3     for metric in registry:                 # ARI, purity, silhouette, ...
4         metric.update(y_true, y_pred)
5     model.learn_one(x)
6     if i % WINDOW == 0:
7         snapshot(registry, step=i)         # one row per window
    
```

Takeaway. Scoring strictly before `learn_one` is what makes the protocol prequential; reversing the two lines would leak the current point into the model and inflate every metric.

3.10 Observations

- **Parameter sensitivity.** `CluStream` reacts strongly to the micro-cluster count q :
 - $q = 50$ — clusters merge too early, ARI drops by 0.09;
 - $q = 200$ — quality plateaus, memory grows linearly;
 - $q = 100$ — chosen default, best quality/memory ratio.
- **Drift response.** Recovery after an abrupt drift takes ≈ 3 windows for `CluStream` and ≈ 7 for `DenStream`.
- **Reproducibility.** All runs are seeded; re-running `make bench` reproduces Table 4 bit-for-bit.

Design decision

Silhouette is computed on a reservoir sample of 2000 points per window instead of the full window. Exact computation is $O(n^2)$ and would dominate the runtime of the benchmark itself, defeating the purpose of the throughput metric.

4. Issues

I1 — Fading factor reacts too slowly after abrupt drift

Type: Technical / algorithmic.

Impact: **MEDIUM** — biases the drift-recovery numbers planned for week 5, but does not block the current phase.

Handling: Reproduced with a minimal script ([issue #7](#)); tested $\lambda \in \{0.25, 0.5, 1.0\}$ and confirmed the effect is parameter-driven, not a bug in River.

Mentor support: Not needed yet. I will propose a fixed λ grid in the week-4 report and only escalate if the grid cannot separate algorithm behaviour from parameter behaviour.

I2 — Ambiguity about which drift definition the proposal targets

Type: Conceptual.

Impact: **HIGH** — the choice between real and virtual drift changes which datasets belong in the benchmark suite.

Handling: Read Gama et al. (2014) and drafted two candidate scopes with the datasets each one implies ([notes/drift-scope.md](#)).

Mentor support: **Yes** — a 10-minute decision at the next sync would unblock the dataset list for Phase 2. My recommendation: cover real drift only, and treat virtual drift as a stretch goal.

Table 5: Issue summary.

ID	Short description	Type	Impact	Mentor input?
I1	Slow fading response after drift	Technical	MEDIUM	No
I2	Drift definition scope unclear	Conceptual	HIGH	Yes

Takeaway. Only I2 needs a decision from the mentors; everything else has an owner and a plan, so the sync can stay short.

5. Next Week's Plan

Table 6: Plan for week 4 (Phase 1 — Benchmark foundation → Phase 2 hand-off).

#	Task	Definition of done (verifiable)	Phase
N1	Finish incremental silhouette	Approximation error < 0.02 vs. exact score on 3 datasets, documented in the notebook	P1.2
N2	Unit tests + CI	$\geq 90\%$ line coverage on <code>bench/metrics.py</code> , GitHub Actions green	P1.2
N3	Wire the full pipeline	<code>make bench</code> runs end-to-end on 1 dataset and emits a CSV	P1.3
N4	Noise-robustness experiment	Table of ARI at 0/5/10% label noise for 3 algorithms	P2.1

Takeaway. N3 is the milestone that closes Phase 1; N1 and N2 are the carry-over items from this week and are scheduled first so the slip does not compound.

Catch-up plan

- The two unfinished items (N1, N2) are front-loaded to Mon–Tue, before any new Phase-2 work starts.
- Estimated catch-up cost: 5h, absorbed inside the normal weekly budget — no schedule extension required.

Risks for next week

- If I2 is not resolved at the sync, N4 will be run on synthetic data only and re-run later once the scope is fixed.

6. References / Links

Code produced this week

Ref	Description	Link
PR #12	Metric suite + registry	pull/12
a1b2c3d	River internals write-up	commit
feat/metrics	Working branch	branch
Notebook	Metric sanity checks	nb/01-metrics.ipynb

Datasets

- **SEA** (synthetic, 100k pts) — generated with `river.datasets.synth.SEA`, seed 42.
- **Coverttype** (581k pts) — UCI Machine Learning Repository, archive.ics.uci.edu/dataset/31.
- **INSECTS** (abrupt variant) — USP data stream repository.

Literature and documentation

References

- [1] J. Gama et al., “A survey on concept drift adaptation,” *ACM Computing Surveys*, 46(4), 2014.
- [2] C. C. Aggarwal et al., “A framework for clustering evolving data streams,” *VLDB*, 2003.
- [3] F. Cao et al., “Density-based clustering over an evolving data stream with noise,” *SDM*, 2006.
- [4] River documentation, *Clustering module*, <https://riverml.xyz>.